CGE-P LAB CURRICULUM · V2

Capstone Project Brief

You've reached the capstone. Everything you've learned (IaC, policy as code, CI/CD, evidence management, OSCAL) comes together in one GitHub repo you build yourself.

You ship the repo, you pass the capstone.

For reading. Code lines wrap to fit the page; copy code from the repository, not the PDF.

GRC Engineering Club

grcengclub.com · based on GRCEngClub/cgep-labs

FROM CHECKBOX GRC TO ENGINEERED SYSTEMS

This PDF is for reading. Long code lines wrap to fit the page, so copy commands and code from `docs/07_01_capstone_brief.md` in your clone, not from the PDF.

The on-ramp: labs are optional, encouraged, and the same skills

The capstone tests the skills the chapter labs taught. The labs are optional and strongly encouraged. Every lab produces an artifact you carry directly into your capstone repo.

If you skipped the labs you can still pass. This brief assumes you didn't. Read "What you've already built" at the bottom before deciding.

One exception, and it is not optional in practice: Lab 2.2, the remote state backend. Layer 3 requires the pipeline to apply on merge to `main`. A GitHub Actions runner with local state has no state, so the first apply after a merge either collides with your existing resources or builds a second copy of your infrastructure. If you skip every other lab, do 2.2. It takes twenty minutes and Layer 3 is not completable without it.

The system

You are the first GRC engineer at **Acme Health**, a 50-person telehealth company. Engineering shipped a Patient Intake API. It works. It is not audit-defensible. The CTO wants it audit-defensible in 30 days, without slowing engineering down.

The starter is real code in a real repo: `GRCEngClub/cgep-app-starter`. Fork it, deploy it, then govern it.

Step zero, the deploy gate:

```
git clone https://github.com/GRCEngClub/cgep-app-starter
cd cgep-app-starter
make deploy AWS_PROFILE=cgep
make test   AWS_PROFILE=cgep
```

If `make test` returns `{"submission_id": "...", "status": "received"}`, you have something to govern. If not, fix that first. Real GRC engineers inherit working systems; demonstrating you can stand one up is the floor.

The brief

Acme is pursuing three flags at once: **HIPAA Security Rule** (PHI is at stake), **SOC 2 Type II** (an enterprise customer is asking), and **CMMC Level 2** (a federal pilot is on the table). You will not satisfy all three. Pick one as your **primary framework**, defend the choice in your write-up, and structure every layer below around it.

The starter ships with eight named, intentional gaps. Design and build a system around it that closes them and produces evidence that the system stays closed.

See `GAPS.md` and `FRAMEWORKS.md` in the starter.

Four layers, one repo

Layer 0: the backend (twenty minutes, and everything depends on it)

An S3 state backend with KMS encryption, versioning, and locking. Lab 2.2. Every workspace below stores state here with its own `key`.

This is not scored as its own layer. It is scored by everything failing without it.

Layer 1: Terraform baseline (around the starter)

The starter gives you the workload. You add the GRC baseline that makes it defensible.

Required new resources:

- **KMS key(s) you own, with rotation enabled.** Bring the starter's S3 uploads bucket and DynamoDB table under your CMK.
- **S3 evidence bucket with Object Lock** (COMPLIANCE or GOVERNANCE, your call, defend it). Versioned, encrypted with your KMS key. Every pipeline run lands here.
- **CloudTrail** trail: multi-region, log-file validation on, writing management events to a dedicated bucket, and **S3 data events scoped to the evidence vault**.
- **Required hardening overrides** on the starter's resources to close `GAPS.md` (for example `aws_s3_bucket_server_side_encryption_configuration` with `sse_algorithm = "aws:kms"`, `aws_lambda_function.intake.vpc_config`, IAM tightened from `dynamodb:*`).

Every S3 bucket you create or harden must meet the curriculum baseline: CMK encryption, versioning, all four public-access-block flags, ACLs disabled, a bucket policy denying `aws:SecureTransport = false`, and lifecycle rules. That is Lab 2.3's `compliant-s3` pattern applied to buckets you did not create.

Use the starter's VPC. Do not build a second one. Closing GAP-05 means moving the Lambda *into* the VPC the starter already created.

Don't pad. A small Terraform that closes five gaps cleanly beats a large one that adds resources without governing the starter.

Layer 2: OPA policy suite

Five or more Rego policies enforcing controls from your **declared primary framework**.

Each policy:

- Has a metadata block naming the framework, control ID(s), severity, and remediation.
- Has its own `_test.rego` with passing and failing fixtures.

- Catches a real gap from `GAPS.md`. Not a generic tag check.
- Cites the control ID in the deny message so a developer reading the failed PR knows what to fix.

New requirement: your suite must be mutation-tested. Commit `scripts/mutation-test.sh` from Lab 3.3 and its output. A policy no test constrains is a policy that cannot fail, and a gate full of those passes every check while enforcing nothing. We will look for this.

New requirement: your gate must fail closed when it has nothing to evaluate. We run your pipeline with the policy directory removed. If it reports success, Layer 2 fails regardless of how good the policies are. Lab 3.4's empty-results guard is the fix.

Conftest runs the suite against your plan in the pipeline. We will also run your policies against a copy of the starter with one of your fixed gaps re-introduced, and confirm the gate fires.

Layer 3: GitHub Actions pipeline

One workflow. Five named steps, in order:

1. **Plan** the Terraform.
2. **Policy check** with Conftest.
3. **Apply** on merge to `main`.
4. **Sign** the evidence bundle with Cosign (keyless, GitHub OIDC).
5. **Upload** the signed bundle to the evidence vault from Layer 1.

Two pull requests must exist in your history: one that passed and merged, one that failed the policy gate and was blocked. Both are evidence the gate works.

Three requirements that separate a working pipeline from a demo:

- **Branch protection must be on, with `enforce_admins: true`.** A red check anyone can merge past is a suggestion, not CM-3. Include the `gh api` output or a screenshot.
- **Plan and apply must use different IAM roles.** The plan role is read-only plus state access. The apply role is scoped to what it changes and trusted only from `repo:OWNER/REPO:environment:production`, behind a GitHub environment protection rule. This is the single highest-value paragraph in most write-ups.
- **Actions pinned by commit SHA**, not by tag. Tags move.

Layer 4: OSCAL component

One `component-definition.json` describing what you actually built, the starter plus the controls you wrapped around it.

- Real v4 UUIDs.
- `control-implementation.source` points at your declared framework's catalog, **anchored to a tag, not `main`**.
- Implementation statements reference real Terraform addresses or ARNs as `props`.
- Evidence links resolve to real signed objects in your vault, by `versionId`.

- A profile selecting the controls your component implements.

New requirement: honest `implementation-status`. Use `implemented`, `partial`, and `inherited` accurately. A control you enforce with code is `implemented`. A control you have tooling for but no scheduled review is `partial`. A control the cloud provider supplies is `inherited`, with the provider named. A component claiming twelve `implemented` controls where three are really `partial` is the copy-paste OSCAL this brief has always warned about, just harder to spot.

New requirement: commit `scripts/verify-oscal-graph.sh` from Lab 6.1, and its output. `trestle validate` proves your document is well-formed. It does not prove a single href resolves. We check both.

If the OSCAL describes a system you didn't build, or cites a framework whose catalog you didn't declare, the layer fails.

The output

A working evidence vault. Every push to `main` produces a signed, timestamped artifact in immutable storage, automatically. That is the whole point.

Three deliverables, due in 30 days

Artifact	Format	What it proves
Public GitHub repo	Terraform + Rego + YAML	You can build the pipeline end to end.
Evidence bundle	Signed `.tar.gz` in S3	Your pipeline produces audit-grade evidence.
Write-up	`WRITEUP.md`	You can explain your design to a stakeholder.

Submission is the repo URL plus the commit SHA you want graded.

The write-up is not optional. Sections: design decisions, control coverage, trade-offs, what you'd do with another sprint, what you didn't get to. **Honest gaps don't lose points. Hand-waving does.**

Three things we score hard

1. **End-to-end integration.** Open a PR, the gate runs, the gate decides whether apply happens, apply triggers signing, signing uploads to the vault. Not four disconnected demos.
2. **Working evidence pipeline.** We pull a recent run and verify it: Cosign signature against the public Sigstore log **with a certificate identity that actually constrains the signer**, SHA-256 recompute, and Object Lock retention. All three must hold. A verify script using `--certificate-identity-regexp '.*'` accepts a bundle signed by any repository on GitHub and does not count as authenticity.

3. **Clear design reasoning.** Your write-up explains *why* you chose each tool and what trade-offs you accepted. Not just *what*.

Three mistakes to avoid

1. **Too much scope.** Thirty resources cleanly integrated beats two hundred bolted on.
2. **Copy-paste OSCAL.** A file that doesn't describe your system is worse than no file. Authenticity over completeness.
3. **Unsigned evidence.** A plan that isn't signed and immutably stored demonstrates no chain of custody.

And one more, new in v2 because it is the most common way a good submission fails:

1. **Controls you claimed but did not implement.** The most frequent instance is AU-6, audit review and analysis, claimed because logs are being collected. Collecting is AU-3 and AU-11. Review requires something that reads them, on a cadence, with a named reviewer. Claim `partial` and say what's missing. **We would rather read an honest `partial` than a confident `implemented` that falls apart in one question.**

What you decide (and defend)

- **Primary framework**: HIPAA Security Rule, SOC 2 TSC, or CMMC Level 2. Pick one.
- AWS region.
- COMPLIANCE vs GOVERNANCE on the Object Lock vault.
- Whether the pipeline applies on merge to `main`, or after a manual approval gate post-merge.
- Single account vs separate evidence-vault account (cleaner: separate; acceptable for 30 days: single).
- Which gaps to close in Terraform vs enforce only in policy. Both valid; defend it.
- **Whether your evidence bundles include raw `terraform state`.** State holds every attribute in plaintext, including secrets, and Object Lock makes an upload undeletable. Lab 2.5 defaults to capturing the plan instead. Whatever you choose, say why in the write-up. A reviewer will ask.

Suggested 30-day plan

- **Week 1 · Design.** Pick your system. Map controls. Sketch the repo. Open a one-page design doc; it becomes the spine of `WRITEUP.md`. **Stand up the Lab 2.2 backend on day one**, before writing any other Terraform, so you never migrate state under a deadline.
- **Week 2 · Build infra.** Terraform baseline. Evidence bucket with Object Lock. KMS. CloudTrail. Apply once by hand from a feature branch. Don't start the pipeline until the baseline applies clean.
- **Week 3 · Policy + pipeline.** Five Rego policies, mutation-tested. The workflow. Signing wired. Open the green PR. Open a second PR that violates a policy and watch it go red. Run the inert-gate test.

- **Week 4 · OSCAL + write-up.** Author the component. Validate with `trestle`, then verify the graph. Wire evidence URIs to real vault objects by version. Write the reflection. Submit.

If you're in week three with no baseline running, cut scope. Trade workload resources for a working pipeline. Every time.

Submission checklist

- [ ] Your repo is a fork or clear derivative of `cgep-app-starter`. The starter's resources are still present and runnable.
- [ ] Remote state backend in use; no `terraform.tfstate` committed except a documented bootstrap.
- [ ] Declared **primary framework** named in `WRITEUP.md`'s first paragraph and in `control-implementation.source`.
- [ ] `terraform/` adds KMS keys, the Object Lock evidence bucket, CloudTrail with data events on the vault, and the gap-closing overrides.
- [ ] Every bucket meets the baseline: CMK, versioning, four PAB flags, ACLs disabled, TLS-deny policy, lifecycle.
- [ ] `policies/` has 5+ Rego policies with tests. `opa test ./policies` passes. Each cites a control ID from your framework.
- [ ] `scripts/mutation-test.sh` committed, output shows every policy killed.
- [ ] The gate fails closed with `policies/` removed. Evidence of that run included.
- [ ] `.github/workflows/grc-gate.yml` runs Plan, Policy check, Apply, Sign, Upload. Actions SHA-pinned.
- [ ] Branch protection on with `enforce_admins: true`.
- [ ] Separate IAM roles for plan and apply.
- [ ] One green PR and one red PR visible in history.
- [ ] At least one signed bundle in the vault. Cosign verifies **against a constrained identity**. SHA matches. Retention active.
- [ ] `oscal/components/<your-component>.json` validates with `trestle`.
- [ ] `scripts/verify-oscal-graph.sh` passes; every evidence href resolves.
- [ ] `WRITEUP.md` covers framework choice, gap remediation, trade-offs, and what you didn't get to.
- [ ] `README.md` is short, with verification instructions for the grader.

What you've already built (if you did the labs)

If you did them, you don't start from scratch, you assemble.

Lab	What it gives you
2.2 Remote State Backend	Layer 0 outright. Without it, Layer 3 cannot be completed.
2.3 First Compliant Resource	Your S3 hardening pattern: CMK, TLS deny, versioning on both buckets, PAB, ACLs off, lifecycle, tags. Drop onto the starter's uploads bucket.
2.4 Modules for Compliance	Module discipline, and the provider-block lesson. Wrap KMS + S3 hardening so dev and prod share one floor.
2.5 IaC as Compliance Evidence	The vault with Object Lock, plus `capture-evidence.sh` and its state-capture guard. The capstone vault IS this vault.
3.3 Writing Rego	Your policy library, the metadata format, the test pattern, and `mutation-test.sh`.
3.4 Conftest + Terraform	`scripts/policy-gate.sh` with the empty-results guard. The pipeline calls it directly.
4.3 GRC Evidence Pipeline	`.github/workflows/grc-gate.yml`, OIDC roles, SHA-pinned actions, branch protection.
4.4 Chain of Custody	Cosign + Object Lock, `verify-evidence.sh` with constrained identity and the completeness check.
5.2 AWS Security Services	CloudTrail with data events, Security Hub, and the Athena queries that make AU-6 claimable.
5.4 GCP Security Services	WIF, if your pipeline touches GCP. The audit-mode-then-enforce rollout answers "without slowing engineering down."
6.1 Introduction to OSCAL	Your component skeleton, the requirements generator, and `verify-oscal-graph.sh`.

If you did the labs as you went, the capstone is one week of design, two weeks of wiring, one week of writing.

If you skipped them, you have 30 days to learn what they teach **and** ship. Doable. Tighter.

Ship the repo. Earn the cert.

Revision history

v2 (current)

- Added Layer 0, the remote state backend. Layer 3's apply-on-merge step is not completable without it.
- Requires the policy suite to be mutation-tested, and requires the gate to fail closed when it loads no policies.
- Requires branch protection with `enforce_admins: true`, and separate IAM roles for plan and apply.
- Requires honest `implementation-status` in the OSCAL, and `verify-oscal-graph.sh` to prove evidence links resolve.

- Cosign verification must constrain the signer identity; a permissive `.*` regex does not demonstrate authenticity.
- Added a fourth common mistake: claiming controls that were collected rather than implemented.

v1

Initial release: four layers, no state backend, no mutation-testing or inert-gate requirement.